Tablas hash con encadenamiento

Pocas claves en un universo enorme

Carlos A Delgado S

Pontificia Universidad Javeriana Cali

Octubre 2026

Objetivos de aprendizaje I

Al finalizar esta sesión, el estudiante podrá

- Calcular la casilla de una clave entera o de una cadena con el método de la división y reconocer cuándo dos claves chocan (OA2).
- Insertar, buscar y eliminar a mano y en C++ en una tabla hash con encadenamiento, donde cada casilla es una lista enlazada (OA2, OA3).
- Dar el costo de cada operación en el peor caso y en el caso promedio en función del factor de carga $\alpha = n/m$, y el espacio que ocupa la tabla (OA1).
- Escoger el número de casillas m y explicar con claves concretas qué pasa con una mala elección (OA1, OA5).

Contenido I

1 La tabla de direccionamiento directo y su límite

2 Pocas claves, universo enorme

3 La función hash

4 Encadenamiento

5 Lo que cuesta

6 Para el juez

7 Ejercicios

Contenido I

1 La tabla de direccionamiento directo y su límite

2 Pocas claves, universo enorme

3 La función hash

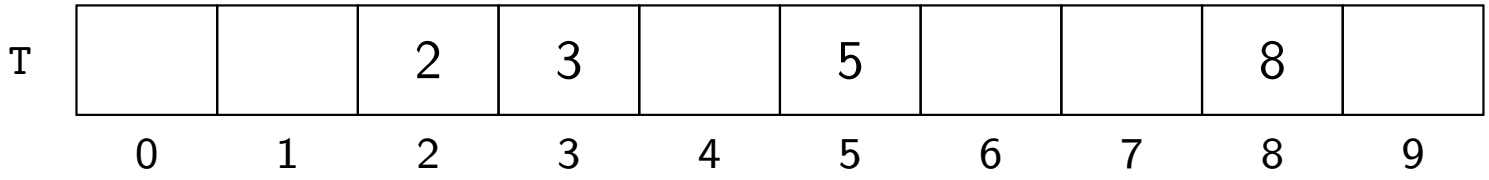
4 Encadenamiento

5 Lo que cuesta

6 Para el juez

7 Ejercicios

Cada clave en su casilla I



La tabla de direccionamiento directo (CLRS, §11.1) guarda la clave k en la casilla $T[k]$. Con claves en $U = \{0, 1, \ldots, u-1\}$:

Operación	Tiempo	Por qué
buscar, insertar, eliminar	$\Theta(1)$	la clave es la dirección
espacio	$\Theta(u)$	una casilla por clave posible, se use o no

Pregunta

Arriba hay 4 claves y 10 casillas. ¿Qué pasa si las claves posibles son 10^{10} y las que de verdad llegan son 40?

Contenido I

1 La tabla de direccionamiento directo y su límite

2 Pocas claves, universo enorme

3 La función hash

4 Encadenamiento

5 Lo que cuesta

6 Para el juez

7 Ejercicios

Un curso de 40 estudiantes I

Problema

Un curso tiene 40 estudiantes y cada uno tiene un código de 10 dígitos. Llegan miles de consultas de la forma "¿el código k está inscrito?". Cada consulta se quiere responder sin recorrer a los 40.

¿Entendimos el problema? I

Entradas

Los n códigos del curso y una lista de consultas.

Salidas

`si` o `no` por cada consulta.

Con tres códigos inscritos, 8920104417, 8920104452 y 8920103990:

Consulta	Respuesta
8920104452	`si`
8920104400	`no`
8920103990	`si`

Pregunta

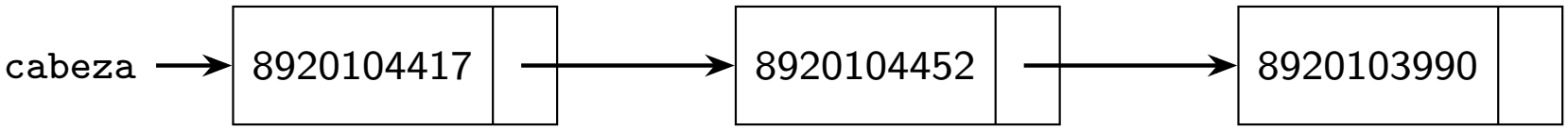
Con la tabla directa, ¿cuántas casillas hacen falta?

¿Entendimos el problema? II

10^{10} casillas

Una casilla por código posible. Aunque cada casilla fuera un solo byte, la tabla ocuparía 10^{10} bytes, unos 10 GB, para guardar 40 códigos. Casi todas quedan vacías.

Primera idea: una lista enlazada I



- Ocupa $\Theta(n)$: un nodo por código inscrito.
- Buscar recorre desde la cabeza: $\Theta(n)$ cuando el código no está. Con q consultas, $\Theta(q \cdot n)$.

	Buscar	Espacio
Tabla directa	$\Theta(1)$	$\Theta(u)$, con $u = 10^{10}$
Lista enlazada	$\Theta(n)$	$\Theta(n)$
Lo que se quiere	cerca de $\Theta(1)$	cerca de $\Theta(n)$

Contenido I

1 La tabla de direccionamiento directo y su límite

2 Pocas claves, universo enorme

3 La función hash

4 Encadenamiento

5 Lo que cuesta

6 Para el juez

7 Ejercicios

Una tabla de 100 casillas I

Tanteo

Se toma una tabla de 100 casillas y cada código va a la casilla que dicen sus dos últimos dígitos.

Código	Casilla
8920104417	17
8920104452	52
8920103990	90

Buscar 8920104452 es mirar la casilla 52: un acceso, como en la tabla directa, con 100 casillas en lugar de 10^{10}.

Pregunta

Se inscribe 8920105517. ¿A qué casilla va?

Una tabla de 100 casillas II

Choque

A la 17, que ya tiene a 8920104417. Dos códigos distintos piden la misma casilla, y la casilla solo guarda uno.

- Hay 10^{10} códigos posibles y 100 casillas: a cada casilla le corresponden 10^8 códigos. Ninguna regla para escoger la casilla evita los choques; lo que se puede hacer es repartirlos bien y saber qué hacer cuando ocurren.

Función hash y colisión I

Definición (CLRS, §11.2)

Una *función hash* $h : U \to \{0, 1, \ldots, m-1\}$ asigna a cada clave del universo U una casilla de una tabla $T[0..m-1]$. La clave k se guarda en $T[h(k)]$, y $h(k)$ es el *valor hash* de k. Dos claves distintas con el mismo valor hash forman una *colisión*.

En el tanteo

U son los códigos de 10 dígitos, $m = 100$ y $h(k)$ son los dos últimos dígitos de k. 8920104417 y 8920105517 colisionan en la casilla 17.

- Si $|U| > m$, dos claves del universo comparten casilla: las colisiones no se pueden evitar del todo.

El método de la división I

Definición (CLRS, §11.3)

$h(k) = k$ mód m: la casilla es el residuo de dividir la clave entre el número de casillas. Los dos últimos dígitos son k mód 100.

Pregunta

Con $m = 7$, ¿a qué casilla va cada clave?

k	k mód 7	$h(k)$
10		
23		
5		
17		

El método de la división II

k	k mód 7	$h(k)$
10	$10 = 1 \cdot 7 + 3$	3
23	$23 = 3 \cdot 7 + 2$	2
5	$5 = 0 \cdot 7 + 5$	5
17	$17 = 2 \cdot 7 + 3$	3

- 10 y 17 colisionan en la casilla 3.
- Calcular $h(k)$ es una división: $\Theta(1)$.
- En C++, `k % m` da el residuo solo si $k \geq 0$. Con claves negativas hay que tener cuidado, y aparece en el problema del juez.

Claves que son palabras I

Problema

Para contar palabras de un texto, la clave es una cadena. Hay que convertirla en un número antes de tomar el residuo.

Primer intento: sumar los códigos de los caracteres

En C, cada `char` es un número: `'s'` es 115, `'o'` es 111, `'l'` es 108. Con $m = 13$: $115 + 111 + 108 = 334$ y 334 mód $13 = 9$.

Pregunta

¿A qué casilla va `los`?

Claves que son palabras II

También a la 9

`los` tiene los mismos caracteres en otro orden y la suma no ve el orden. Toda palabra choca con sus anagramas: `gato` y `toga`, `amor` y `roma`.

Leer la cadena como un número en base b

Así como $417 = (4 \cdot 10 + 1) \cdot 10 + 7$, la cadena $c_0 c_1 \cdots c_{L-1}$ se lee como $(\cdots((c_0 \cdot b + c_1) \cdot b + c_2) \cdots) \cdot b + c_{L-1}$, y la casilla es ese número módulo m (CLRS, §11.3). Cada carácter pesa distinto según su posición. Aquí $b = 31$.

El polinomio, paso a paso I

Con $b = 31$ y $m = 13$, se reduce módulo 13 en cada paso:

Paso	Carácter	Código	$h \leftarrow (h \cdot 31 + c)$ mód 13
0			$h = 0$
1	`s`	115	$(0 \cdot 31 + 115)$ mód $13 = 11$
2	`o`	111	$(11 \cdot 31 + 111)$ mód $13 = 452$ mód $13 = 10$
3	`l`	108	$(10 \cdot 31 + 108)$ mód $13 = 418$ mód $13 = 2$

Pregunta

¿Por qué reducir en cada paso y no al final?

El polinomio, paso a paso II

- Sin reducir, el número crece como 31^L. Con `'a'` $= 97$, una palabra de seis letras ya pasa de $97 \cdot 31^5 \approx 2{,}8 \cdot 10^9$, y un `int` llega hasta $2^{31} - 1 \approx 2{,}1 \cdot 10^9$. Reduciendo, $h < m$ siempre, y el residuo da el mismo resultado al final.

```
// Lee la cadena como un numero en base 31 (Horner), reduciendo en
↪  cada paso.
int hashPolinomio(const char *s, int m) {
  int h = 0;
  int i = 0;
  while (s[i] != '\0') {
    h = (h * 31 + s[i]) % m;
    i = i + 1;
  }
  return h;
}
```

El polinomio, paso a paso III

El programa calcula las dos funciones con $m = 13$:

Palabra	Suma	Polinomio
`sol`	9	2
`los`	9	3
`gato`	11	1
`toga`	11	12

- Con el polinomio los anagramas se separan. Otras palabras pueden chocar: con 13 casillas, es inevitable.
- Calcular el valor hash de una cadena de L caracteres cuesta $\Theta(L)$: una vuelta por carácter.

Contenido I

1 La tabla de direccionamiento directo y su límite

2 Pocas claves, universo enorme

3 La función hash

4 Encadenamiento

5 Lo que cuesta

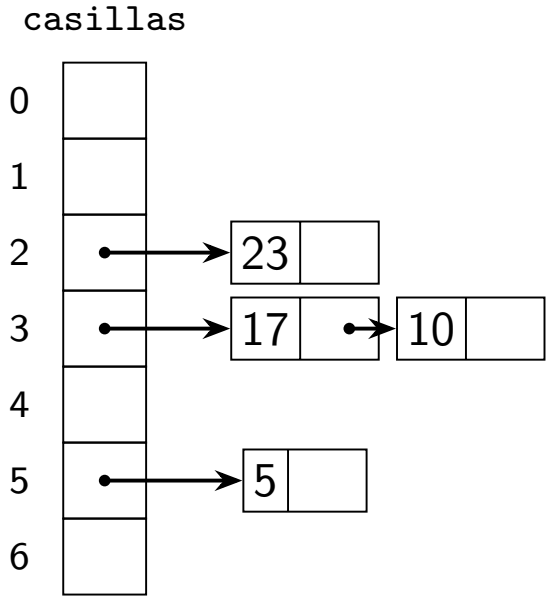
6 Para el juez

7 Ejercicios

Una lista enlazada en cada casilla I

Encadenamiento (CLRS, §11.2)

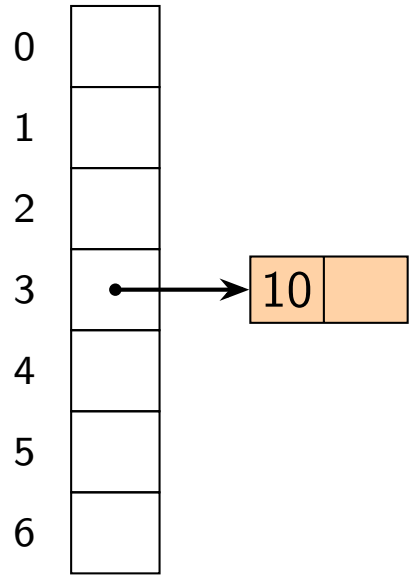
Cada casilla $T[i]$ guarda la cabeza de una lista enlazada con todas las claves k tales que $h(k) = i$. Una casilla sin claves guarda `NULL`. Las colisiones se resuelven en la lista.



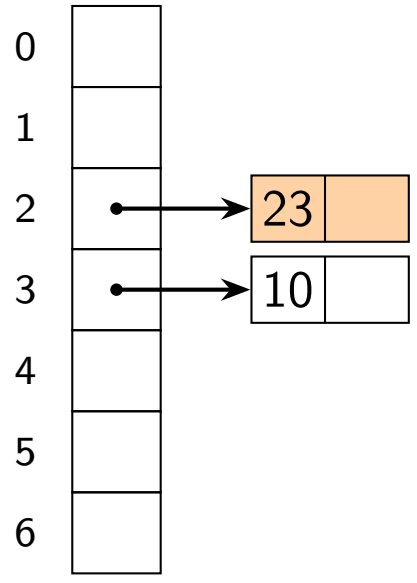
Las claves 10, 23, 5 y 17 con $m = 7$: 10 y 17 comparten la lista de la casilla 3. Cada nodo es el de la lista enlazada simple: la clave y el puntero al siguiente.

Paso a paso: 10, 23, 5, 17, 12, 31 con $m = 7$ I

insertar 10: $h(10) = 3$



insertar 23: $h(23) = 2$

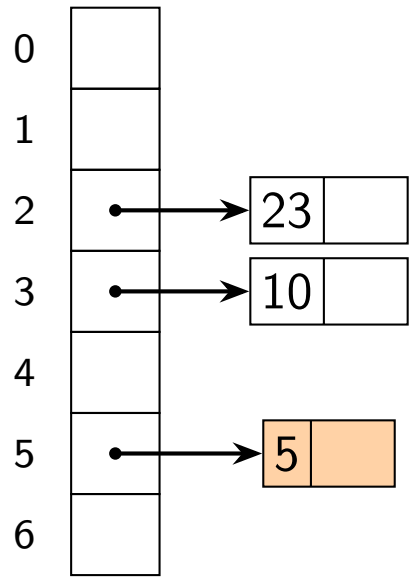


Pregunta

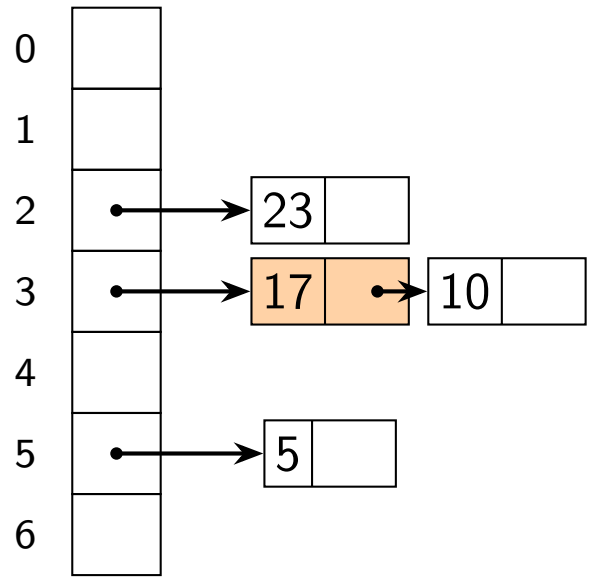
Después de insertar 5, llega 17 y $h(17) = 3$. ¿Va al frente de la lista de la casilla 3 o al final?

Paso a paso: 10, 23, 5, 17, 12, 31 con $m = 7$ II

insertar 5: $h(5) = 5$



insertar 17: $h(17) = 3$

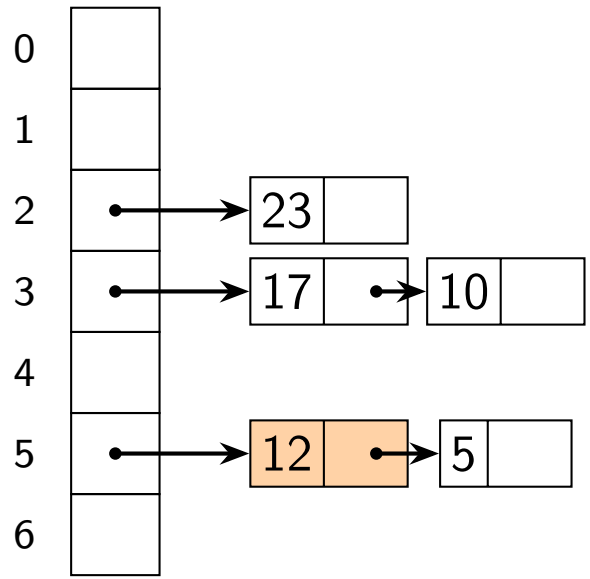


Al frente

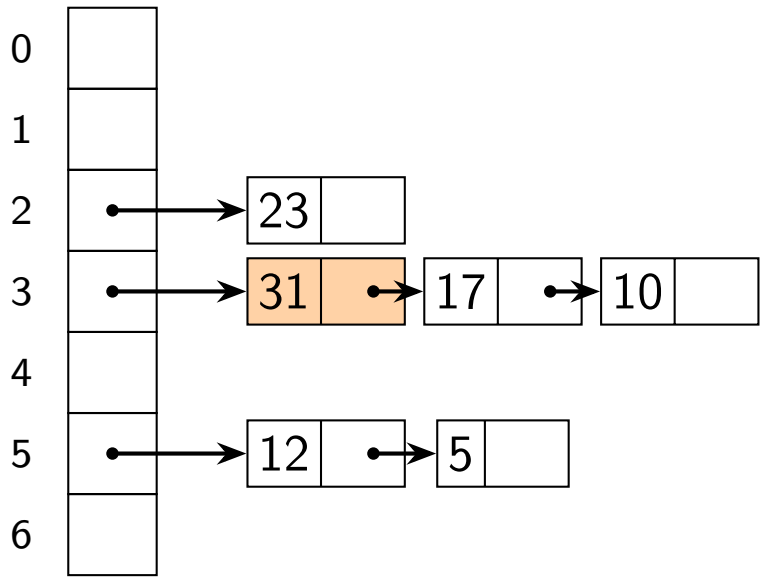
El 17 se cuelga antes del 10: el nuevo nodo apunta a la cabeza vieja y la casilla apunta al nuevo nodo. Son dos punteros, sin importar el largo de la lista. Ir al final pediría recorrerla.

Paso a paso: 10, 23, 5, 17, 12, 31 con $m = 7$ III

insertar 12: $h(12) = 5$



insertar 31: $h(31) = 3$

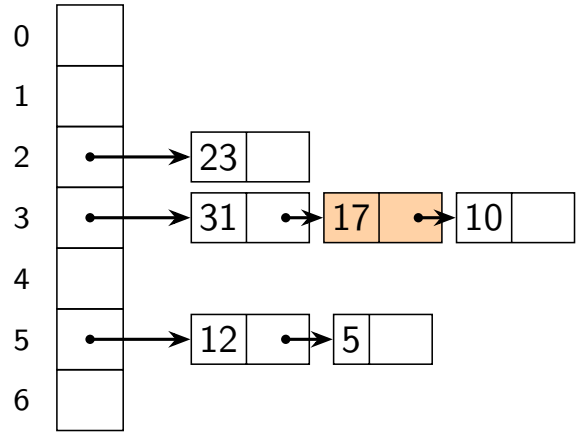


Paso a paso: 10, 23, 5, 17, 12, 31 con $m = 7$ IV

Lo que pasó en cada inserción:

Clave	$h(k)$	Largo de su lista	Casillas vacías
10	3	1	6
23	2	1	5
5	5	1	4
17	3	2	4
12	5	2	4
31	3	3	4

Buscar I



Buscar 17

$h(17) = 3$. Se recorre la lista de la casilla 3: 31 no es, 17 sí. Dos comparaciones. Las listas de las casillas 2 y 5 no se miran.

Pregunta

Buscar 24. ¿En qué casilla se busca y cuándo se sabe que no está?

Buscar II

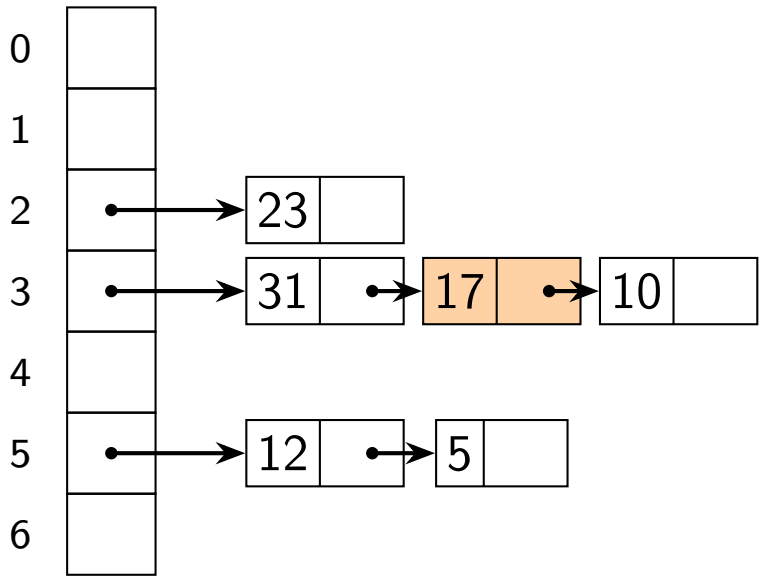
Buscar 24

$h(24) = 24 \text{ mód } 7 = 3$. Se compara con 31, 17 y 10, y se llega a `NULL`: 24 no está. Tres comparaciones, el largo entero de la lista.

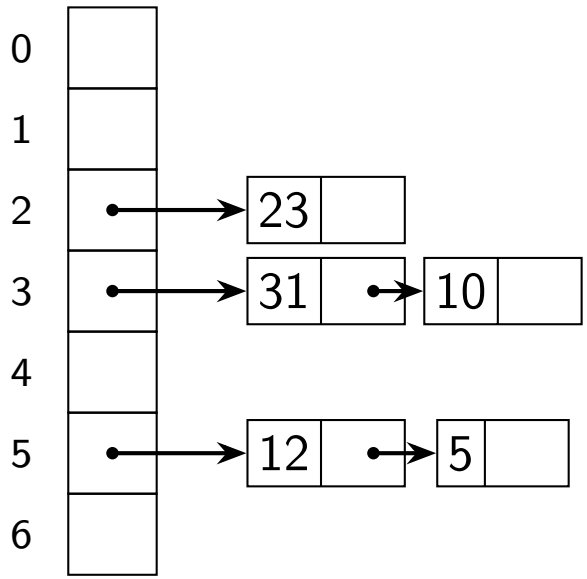
- Una búsqueda que falla recorre la lista completa de su casilla. Una que acierta se detiene en la clave.
- El costo depende del largo de una lista, no del total de claves de la tabla.

Eliminar I

antes: `anterior` en 31



después de eliminar 17



Eliminar II

Eliminar 17 es eliminar en la lista enlazada: se recorre la lista de la casilla 3 con dos punteros, `anterior` y `actual`. Cuando `actual` llega al 17, el `siguiente` de 31 pasa a ser el 10 y el nodo del 17 se libera con `delete`.

Caso límite

Si la clave está en la cabeza, no hay `anterior`: la casilla misma pasa a apuntar al segundo nodo. Eliminar 31 deja `casillas[3]` en el 17.

¿Por qué no un arreglo de `Lista`? I

Pregunta

El TAD Lista enlazada ya está construido. ¿No basta con `Lista casillas[7]`, insertar con `insertar(0, k)` y buscar con `obtener(p)`?

```
int p = 0;
while (p < casillas[i].tamano() && casillas[i].obtener(p) !=
↪  k) {
  p = p + 1;
}
```

¿Por qué no un arreglo de `Lista`? II

Buscar se vuelve cuadrático

`obtener(p)` camina desde la cabeza hasta la posición p. Para revisar una lista de largo L se hacen $0 + 1 + \cdots + (L-1) = L(L-1)/2$ pasos: $\Theta(L^2)$ en lugar de $\Theta(L)$.

- `insertar(0, k)` sí cuesta $\Theta(1)$.
- La tabla recorre sus nodos con un puntero que avanza, como el iterador de `list`: un paso por nodo.

La tabla en C++ I

```
struct Nodo {
  int clave;
  Nodo *siguiente;
};

class TablaHash {
private:
  Nodo **casillas;  // cabeza de la lista de cada casilla
  int m;            // numero de casillas
  int n;            // numero de claves guardadas

  // Metodo de la division. Exige k >= 0.
  int h(int k) {
    return k % m;
  }
```

La tabla en C++ II

```
public:
  TablaHash(int tam) {
    m = tam;
    n = 0;
    casillas = new Nodo *[m];
    int i = 0;
    while (i < m) {
      casillas[i] = NULL;
      i = i + 1;
    }
  }
```

El constructor deja las m cabezas en `NULL`: $\Theta(m)$.

La tabla en C++ III

```
// Recorre la lista de h(k) hasta hallar k o llegar a NULL.
bool buscar(int k) {
  Nodo *actual = casillas[h(k)];
  while (actual != NULL && actual->clave != k) {
    actual = actual->siguiente;
  }
  return actual != NULL;
}
```

```
// Cuelga k al frente de su lista. Exige que k no este.
void insertar(int k) {
  int i = h(k);
  Nodo *nuevo = new Nodo;
  nuevo->clave = k;
  nuevo->siguiente = casillas[i];
  casillas[i] = nuevo;
  n = n + 1;
}
```

La tabla en C++ IV

```
// Desenlaza el nodo de k. Devuelve si lo encontro.
bool eliminar(int k) {
  int i = h(k);
  Nodo *anterior = NULL;
  Nodo *actual = casillas[i];
  while (actual != NULL && actual->clave != k) {
    anterior = actual;
    actual = actual->siguiente;
  }
  bool estaba = actual != NULL;
  if (estaba) {
    if (anterior == NULL) {
      casillas[i] = actual->siguiente;
    } else {
      anterior->siguiente = actual->siguiente;
    }
    delete actual;
    n = n - 1;
  }
  return estaba;
}
```

La tabla en C++ V

```
~TablaHash() {
  int i = 0;
  while (i < m) {
    while (casillas[i] != NULL) {
      Nodo *muerto = casillas[i];
      casillas[i] = casillas[i]->siguiente;
      delete muerto;
    }
    i = i + 1;
  }
  delete[] casillas;
}
```

- El destructor libera nodo por nodo cada lista y después el arreglo de cabezas: cada `new` tiene su `delete` y el `new[]` su `delete[]`.
- `insertar` no revisa si la clave ya está. Si las claves pueden repetirse, se llama a `buscar` antes.

El ejemplo, corriendo I

```
int main() {
  TablaHash t(7);
  int claves[6] = {10, 23, 5, 17, 12, 31};
  int i = 0;
  while (i < 6) {
    printf("h(%d) = %d\n", claves[i], claves[i] % 7);
    t.insertar(claves[i]);
    i = i + 1;
  }
  t.imprimir();
  printf("buscar 17: %d\n", t.buscar(17));
  printf("buscar 24: %d\n", t.buscar(24));
  printf("eliminar 17: %d\n", t.eliminar(17));
  printf("largo de la casilla 3: %d\n", t.largo(3));
  printf("n = %d, m = %d\n", t.tamano(), t.casillasTotal());
  return 0;
}
```

El ejemplo, corriendo II

Salida

`h(10) = 3`, `h(23) = 2`, `h(5) = 5`, `h(17) = 3`, `h(12) = 5`, `h(31) = 3`, y la tabla:

```
0:  1:  2: 23  3: 31 17 10  4:  5: 12 5  6:
```

Después, `buscar 17: 1`, `buscar 24: 0`, `eliminar 17: 1`, `largo de la casilla 3: 2` y `n = 5, m = 7`.

Las listas salen como en los dibujos: la última clave insertada en cada casilla queda de primera.

Contenido I

1 La tabla de direccionamiento directo y su límite

2 Pocas claves, universo enorme

3 La función hash

4 Encadenamiento

5 Lo que cuesta

6 Para el juez

7 Ejercicios

El factor de carga I

Definición (CLRS, §11.2)

Con n claves en una tabla de m casillas, el *factor de carga* es $\alpha = n/m$: el largo promedio de las listas.

En el ejemplo

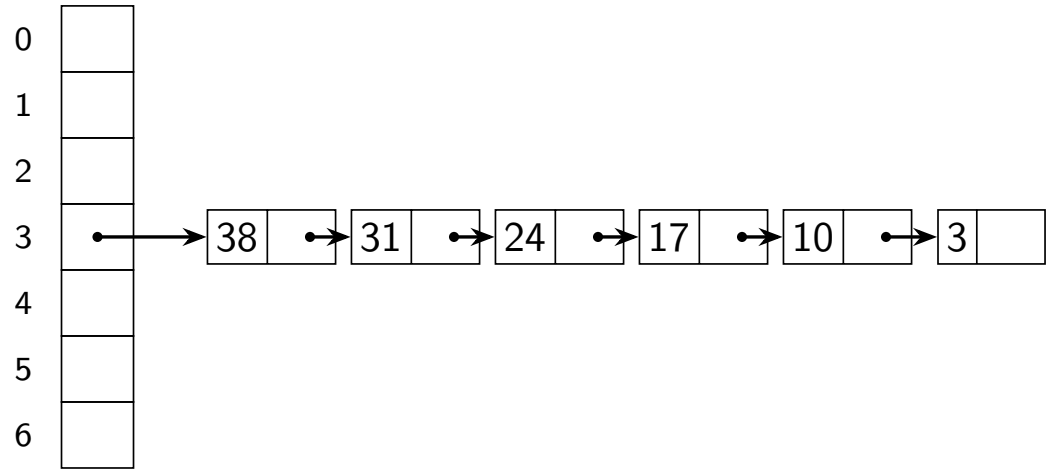
Con las seis claves, $\alpha = 6/7 \approx 0{,}86$. Las listas miden 0, 0, 1, 3, 0, 2 y 0: suman 6, y su promedio es $6/7$. Tras eliminar 17, $\alpha = 5/7 \approx 0{,}71$.

Pregunta

α es el promedio. ¿Cuánto puede medir la lista más larga?

El peor caso: todas a la misma casilla I

Las claves 3, 10, 17, 24, 31 y 38 con $m = 7$: todas dejan residuo 3.



- Seis casillas vacías y una lista con las $n = 6$ claves. α sigue siendo $6/7$.
- Buscar 45, con $h(45) = 3$, compara con las seis antes de llegar a `NULL`. La tabla se comporta como la lista enlazada del comienzo: buscar y eliminar cuestan $\Theta(n)$.
- Insertar al frente sigue costando $\Theta(1)$.

El caso promedio I

Hashing uniforme independiente (CLRS, §11.2)

Se supone que cada clave cae en cualquiera de las m casillas con la misma probabilidad, sin importar dónde cayeron las demás.

- Con ese supuesto, la lista de una casilla mide en promedio α. Una búsqueda que falla recorre una lista entera; una que acierta, en promedio, cerca de la mitad de una lista más el nodo de la clave.
- CLRS (teoremas 11.1 y 11.2) concluye que las dos cuestan, en promedio, $\Theta(1+\alpha)$: el 1 es calcular $h(k)$ y llegar a la casilla, y α es el recorrido.

Pregunta

Si n crece y m se deja fijo, ¿qué pasa con $1+\alpha$? ¿Y si m crece al mismo ritmo que n?

El caso promedio II

Con m fijo, $\alpha = n/m$ crece con n y buscar vuelve a ser lineal, solo que m veces más rápido que la lista. Con m proporcional a n, el promedio queda acotado:

Teorema

Si la tabla tiene $m \geq n/2$ casillas, entonces $f(n) = 1 + n/m$ es $O(1)$.

Demostración

Se procede de forma directa: se busca una constante c y un k tales que $f(n) \leq c \cdot 1$ para todo $n \geq k$.

El caso promedio III

Desarrollo

$$
\begin{aligned}
f(n) &= 1 + \frac{n}{m} && \text{definición de } \alpha \\
&\leq 1 + \frac{n}{n/2} && m \geq n/2\text{: dividir entre más da menos} \\
&= 1 + 2 = 3 && \frac{n}{n/2} = 2 \text{ para } n \geq 1 \\
&= 3 \cdot 1 && \text{se toma } c = 3
\end{aligned}
$$

La desigualdad vale desde $n = 1$, así que $k = 1$.

Conclusión

Por lo tanto, se puede concluir que $1 + \alpha \in O(1)$ con testigos $c = 3$ y $k = 1$: si la tabla mantiene al menos una casilla por cada dos claves, buscar y eliminar cuestan $O(1)$ en promedio.

Lo que cobra la tabla hash con encadenamiento I

Operación	Promedio	Peor caso	Por qué
$h(k)$, clave entera	$\Theta(1)$	$\Theta(1)$	una división
$h(s)$, cadena de L	$\Theta(L)$	$\Theta(L)$	una vuelta por carácter
`insertar(k)`	$\Theta(1)$	$\Theta(1)$	se cuelga al frente de su lista
`buscar(k)`	$\Theta(1+\alpha)$	$\Theta(n)$	se recorre una lista; en el peor caso, todas las claves están en ella
insertar sin repetir	$\Theta(1+\alpha)$	$\Theta(n)$	primero se busca
`eliminar(k)`	$\Theta(1+\alpha)$	$\Theta(n)$	se busca con `anterior`; desenlazar es $\Theta(1)$
`largo(i)`	$\Theta(1+\alpha)$	$\Theta(n)$	se cuentan los nodos de una lista
`imprimir`, destructor	$\Theta(m+n)$	$\Theta(m+n)$	se visitan las m casillas y los n nodos
constructor	$\Theta(m)$	$\Theta(m)$	m cabezas en `NULL`

Lo que cobra la tabla hash con encadenamiento II

Espacio

$\Theta(m+n)$: el arreglo de m cabezas y un nodo por clave, con la clave y un puntero. Ninguna operación pide memoria aparte que crezca con n.

	Buscar, promedio	Buscar, peor caso	Espacio
Lista enlazada	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$
Tabla directa	$\Theta(1)$	$\Theta(1)$	$\Theta(u)$
Tabla hash, m proporcional a n	$O(1)$	$\Theta(n)$	$\Theta(n)$

Escoger m I

Pregunta

Se insertan las 1000 claves $4, 8, 12, \ldots, 4000$ en una tabla de $m = 64$ casillas. $\alpha = 1000/64 \approx 15{,}6$. ¿Cuánto mide la lista más larga?

Para responder, un programa llena la tabla y cuenta los largos. `llenar(t, paso)` inserta las claves $paso \cdot 1, \ldots, paso \cdot 1000$, y `informe` recorre las m casillas con `largo`.

Escoger *m* II

```
void informe(int m, int paso) {
  TablaHash t(m);
  llenar(t, paso);
  int vacias = 0;
  int mayor = 0;
  int j = 0;
  while (j < m) {
    int l = t.largo(j);
    if (l == 0) {
      vacias = vacias + 1;
    }
    if (l > mayor) {
      mayor = l;
    }
    j = j + 1;
  }
  printf("m %2d paso %2d: vacias %2d, mayor %4d, alfa %.1f\n",
         m, paso, vacias, mayor, 1000.0 / m);
}
```

Escoger m III

El programa inserta $k = paso \cdot i$ con $i = 1, \ldots, 1000$ y cuenta:

m	paso	Casillas vacías	Lista más larga	α
64	1	0	16	15,6
64	4	48	63	15,6
61	4	0	17	16,4
61	61	60	1000	16,4

Escoger m IV

- $64 = 2^6$, y k mód 2^6 son los últimos 6 bits de k. Un múltiplo de 4 termina en 00: solo quedan 16 casillas posibles, las múltiplos de 4, y las otras 48 nunca se usan. Las listas miden cuatro veces α.
- Con el primo 61 los múltiplos de 4 recorren todos los residuos: ninguna casilla vacía y listas de 16 o 17.
- Ningún m resiste un patrón hecho a su medida: con paso 61, las 1000 claves caen en la casilla 0 de la tabla de 61.

Regla (CLRS, §11.3)

Con el método de la división se escoge un primo que no esté cerca de una potencia de 2. Una potencia de 2 o de 10 usa solo los últimos bits o los últimos dígitos de la clave, como la tabla de 100 casillas con los códigos.

Contenido I

1 La tabla de direccionamiento directo y su límite

2 Pocas claves, universo enorme

3 La función hash

4 Encadenamiento

5 Lo que cuesta

6 Para el juez

7 Ejercicios

Para el juez: UVa 484 — The Department of Redundancy Department I

UVa 484 — The Department of Redundancy Department,
`https://onlinejudge.org/external/4/484.pdf`
Envío: `https://onlinejudge.org/index.php?option=com_onlinejudge&Itemid=25&page=submit_problem&problemid=425`

Qué pide

La entrada es una secuencia de enteros positivos, negativos o cero, tan larga como se quiera, hasta el fin del archivo. Por cada entero distinto se imprime una línea con el entero y cuántas veces apareció, en el orden en que apareció por primera vez.

¿Entendimos el problema?

Con 3 1 2 2 1 3 5 3 3 2 sale 3 4, 1 2, 2 3 y 5 1. El 3 va primero porque fue el primero en llegar, no porque sea el que más se repite.

Para el juez: UVa 484 — The Department of Redundancy Department II

Cómo atacarlo con lo de hoy

Una tabla hash con encadenamiento cuyo nodo guarda la clave y un contador. Por cada número se busca en su lista: si está, se suma uno; si no, se cuelga un nodo con cuenta 1 y el número se anota en un arreglo de claves en orden de llegada. Al final se recorre ese arreglo y se busca la cuenta de cada una. Con $m = 100\,003$, primo, cada número cuesta $\Theta(1 + \alpha)$ en promedio.

Las trampas

- Los negativos: en C++, `-7 % 5` da -2. Con `k % m` como casilla, un negativo escribe fuera del arreglo. La casilla es `((k % m) + m) % m`.
- No se sabe cuántos números vienen, así que no hay n para dimensionar: m se fija grande desde el comienzo.
- $-100\,003$, 0 y $100\,003$ caen en la misma casilla y son claves distintas: la lista compara la clave, no la casilla.

Contenido I

1 La tabla de direccionamiento directo y su límite

2 Pocas claves, universo enorme

3 La función hash

4 Encadenamiento

5 Lo que cuesta

6 Para el juez

7 Ejercicios

Ejercicio en parejas: a mano I

Inserten, en este orden, 22, 5, 16, 38, 27, 49 y 3 en una tabla con encadenamiento de $m = 11$ casillas, colgando al frente.

Las preguntas

1. Dibujen la tabla. ¿Cuánto vale α y cuánto mide la lista más larga?
2. ¿Cuántas comparaciones hace buscar 60?
3. Repitan con $m = 10$. ¿Qué tabla queda mejor repartida, y por qué no es la del primo?

Ejercicio en parejas: a mano II

Lo que debe salir

1. Con $m = 11$: 22 va a la casilla 0, 3 a la 3, y 5, 16, 38, 27 y 49 a la 5, con la lista `49 27 38 16 5`. $\alpha = 7/11 \approx 0{,}64$ y la lista más larga mide 5.
2. 60 mód 11 = 5: cinco comparaciones y `NULL`. No está.
3. Con $m = 10$ cada clave va a su propia casilla: 2, 5, 6, 8, 7, 9 y 3. Las claves 5, 16, 27, 38 y 49 van de 11 en 11, un patrón hecho a la medida de $m = 11$. El primo protege contra los patrones comunes, no contra todos.

Ejercicio en parejas: dos que suman x I

Dado un arreglo de n enteros no negativos y un número x, decir si hay dos casillas distintas cuyos valores suman x.

Las preguntas

1. Con $a = \langle 8, 3, 11, 6, 2, 15 \rangle$, ¿qué responde con $x = 9$, $x = 21$ y $x = 12$?
2. ¿Cuánto cuesta probar todas las parejas?
3. Al llegar a $a[i]$, ¿qué número habría que haber visto antes? ¿Qué estructura responde eso rápido?

Ejercicio en parejas: dos que suman x II

```
// Exige a[i] >= 0. Antes de guardar a[i], busca su pareja x - a[i].
bool sumaPar(int a[], int n, int x) {
  TablaHash vistos(11);
  bool hay = false;
  int i = 0;
  while (i < n && !hay) {
    int falta = x - a[i];
    if (falta >= 0 && vistos.buscar(falta)) {
      hay = true;
    }
    if (!vistos.buscar(a[i])) {
      vistos.insertar(a[i]);
    }
    i = i + 1;
  }
  return hay;
}
```

Ejercicio en parejas: dos que suman x III

Lo que debe salir

1. $x = 9$: sí, $3 + 6$. $x = 21$: sí, $6 + 15$. $x = 12$: no; $6 + 6$ no cuenta, porque hay un solo 6. El programa imprime `1`, `1` y `0`.
2. Todas las parejas son $n(n-1)/2$: $\Theta(n^2)$.
3. $x - a[i]$. Se busca en la tabla de los ya vistos y después se guarda $a[i]$. Buscar antes de guardar es lo que impide usar el 6 dos veces. Son n búsquedas e inserciones de $\Theta(1 + \alpha)$ en promedio: $\Theta(n)$ con m proporcional a n, más $\Theta(n)$ de espacio aparte.

Propuestos para la casa I

1. **Distintos.** Escriba una función que cuente cuántos valores distintos hay en un arreglo de n enteros no negativos con la tabla hash. ¿Cuánto cuesta? ¿Y ordenando primero?
2. **Tabla de palabras.** Cambie la clave de `TablaHash` a una cadena de hasta 20 letras, con `hashPolinomio` como h. ¿Qué cambia en `buscar`?
3. **El peor caso a la medida.** Para $m = 13$, escriba ocho claves que caigan todas en la casilla 4, y cuente cuántas comparaciones hace buscar una clave que no está.
4. **Repartir códigos.** Genere 2000 códigos de 10 dígitos que comparten los seis primeros y mida, con `largo`, la lista más larga y las casillas vacías con $m = 100$, $m = 128$ y $m = 101$.
5. **Eliminar con la lista doble.** Si cada casilla fuera una lista doblemente enlazada y se tuviera el nodo en la mano, ¿cuánto costaría eliminar?

Propuestos para la casa II

Para pensar

Una tabla de $m = 101$ casillas recibe un millón de claves. ¿Qué le pasa a α, y cómo haría la tabla para mantener buscar en $O(1)$ en promedio sin saber n desde el comienzo?

Pistas para los propuestos I

1. Se busca cada valor y, si no estaba, se inserta y se suma uno. $\Theta(n)$ en promedio con m proporcional a n; ordenando, $\Theta(n \log n)$ y un recorrido que compara vecinos.
2. El nodo guarda un arreglo de `char`; la comparación `actual->clave != k` pasa a ser `strcmp`, y cada comparación cuesta lo que la cadena.
3. Sirve cualquier clave $4 + 13j$. Buscar una que no está recorre las ocho.
4. Con 100 casillas solo cuentan los dos últimos dígitos, y con 128, los últimos 7 bits. Compare con el primo.
5. Con `anterior` en cada nodo no hay que buscar al de antes: desenlazar es $\Theta(1)$ (CLRS, §11.2).

Resumen I

- La tabla directa busca en $\Theta(1)$ pero ocupa una casilla por clave posible; con pocas claves en un universo enorme, no cabe.
- Una función hash lleva cada clave a una de m casillas. Con el método de la división, $h(k) = k$ mód m; una cadena se lee como un número en base 31, reduciendo en cada paso.
- Las colisiones son inevitables cuando el universo es mayor que la tabla. Con encadenamiento, cada casilla es la cabeza de una lista enlazada: insertar al frente cuesta $\Theta(1)$, y buscar y eliminar recorren una sola lista.
- Con $\alpha = n/m$, buscar y eliminar cuestan $\Theta(1 + \alpha)$ en promedio y $\Theta(n)$ en el peor caso. El espacio es $\Theta(m + n)$.
- m se escoge primo y lejos de una potencia de 2: con $m = 64$ y claves múltiplos de 4, 48 casillas quedaron vacías.

Resumen II

Lo que sigue

Más tablas hash: qué hacer cuando α crece, y el contenedor `map` de la biblioteca, que guarda pares de clave y valor.

Referencias I



T. H. Cormen, C. E. Leiserson, R. L. Rivest y C. Stein. *Introduction to Algorithms*, 4.ª ed. MIT Press, 2022. Capítulo 11, tablas hash: §11.1 tablas de direccionamiento directo; §11.2 tablas hash, encadenamiento y su análisis; §11.3 funciones hash, método de la división.



R. Thareja. *Data Structures Using C*. Oxford University Press, 2018. Capítulo 15, hashing y colisiones.



UVa Online Judge. Problema 484, *The Department of Redundancy Department*: `https://onlinejudge.org/external/4/484.pdf`.